

Phase 1 — Backend Foundation

Goal: Build the first working version of the backend and core AI logic.

End result: A small cybersecurity log file can be processed into structured events, an event graph, and at least one incident conclusion with supporting evidence.

1. Finalise Input and Event Format

Start with CSV/JSON cybersecurity logs. Define one common Event object for all log sources.

```
class Event(BaseModel):
    timestamp: datetime
    source: str
    event_type: str
    actor: str | None = None
    target: str | None = None
    metadata: dict = {}
```

Deliverable: raw log → standard Event object.

2. Build the Log Parser

Use Python + Pandas to read CSV/JSON, clean data, convert timestamps, map fields, and sort events.

```
df = pd.read_csv("logs.csv")
df["timestamp"] = pd.to_datetime(df["timestamp"])
df = df.sort_values("timestamp")
```

Deliverable: sample dataset successfully converted into valid Event objects.

3. Define Event Types

Start with a small controlled set: process_execution, file_download, file_creation, registry_modification, network_connection, powershell_execution, log_deletion. Keep the list based on the selected dataset.

4. Build Event Relationships

Initially relate events using close timestamps, same process, same file/object, same user, and parent/child process relationships.

```
Download invoice.zip
↓
Extract invoice.zip
↓
Execute setup.exe
```

5. Build the Event Graph

Use NetworkX. Events become nodes and relationships become edges.

```
graph.add_node(event_id, event=event)
graph.add_edge(
    download_id,
    extract_id,
    relation="followed_by"
)
```

Target: construct a meaningful graph from structured events.

6. Build the Cybersecurity Knowledge Base

Research a small set of attack behaviours using sources such as MITRE ATT&CK. For each behaviour document required events, conditions, conclusion, explanation, and source.

```
Rule: Possible Registry Persistence
Required: process_execution + registry_modification
Condition: registry change shortly after execution
Conclusion: possible_persistence
```

7. Implement the Symbolic AI / Reasoning Engine

Use rule-based reasoning + forward chaining.

```
def detect_persistence(events):
    executed = any(e.event_type == "process_execution" for e in events)
```

```

registry = any(e.event_type == "registry_modification" for e in events)

if executed and registry:
    return "possible_persistence"
return None

```

Add temporal conditions later and store the evidence supporting every conclusion.

```

{
    "conclusion": "possible_persistence",
    "evidence": [
        "setup.exe executed at 09:12",
        "Registry Run key modified at 09:13"
    ],
    "rule": "REG-PERSIST-01"
}

```

8. Implement Forward Chaining

```

facts = initial_events

while True:
    new_facts = apply_rules(facts)
    if not new_facts:
        break
    facts.extend(new_facts)

```

Example: execution + registry modification → possible persistence. New conclusions can become facts for later rules.

9. Add FastAPI

Expose the working backend through a small API.

```

POST /upload
POST /analyze
GET /incident/{id}

@app.post("/analyze")
async def analyze(file: UploadFile):
    events = parse_logs(file)
    graph = build_graph(events)
    result = reason(graph)
    return result

```

10. Testing

Create small tests from the start:

- Parser: 10 rows → 10 valid Events
- Graph: A → B → C → 3 nodes, 2 edges
- Rule: Execution + Registry Modification → Possible Persistence
- Negative case: Normal activity → No persistence conclusion

Phase 1 Deliverables

- Selected sample cybersecurity dataset
- Common Event schema
- CSV/JSON parser
- Event classification
- Event relationship logic
- NetworkX event graph
- Initial cybersecurity rules
- Forward-chaining reasoning prototype
- Evidence attached to conclusions
- Basic FastAPI backend
- End-to-end backend test

Definition of Done

attack_sample.csv
↓
/analyze
↓
Structured Events
↓
Event Graph
↓
Reasoning
↓
incident = possible_persistence
evidence = supporting events
rule = REG-PERSIST-01

Phase 1 is complete when this backend pipeline works end-to-end on a sample dataset.